Residency Day 3 Project 4: Secure Online Communication with TLS

Group 2

Bilal Khalid

Nischal Pokharel

Samia Zaman

Syed Kashif Ali

Steven Sisjayawan

Amer Mohammed

University of the Cumberlands

MSCS-535 Secure Software Development

2026 Spring

Prof. Brandon Bass

March 28, 2026

Abstract

Secure online communication is a fundamental requirement in modern distributed systems, particularly in the presence of adversaries capable of intercepting or modifying network traffic. The Transport Layer Security (TLS) protocol provides confidentiality, integrity, and authentication through cryptographic mechanisms. This report presents an overview of TLS and demonstrates pseudocode for a secure communication system designed to mitigate man-in-the-middle (MITM) attacks. The discussion emphasizes certificate validation, authenticated key exchange, and session encryption as core defenses.

Introduction

Transport Layer Security (TLS) is a cryptographic protocol designed to secure communication over computer networks. It evolved from Secure Sockets Layer (SSL) and is widely used in protocols such as HTTPS to ensure confidentiality, integrity, and authenticity of data exchanged between clients and servers (Rescorla, 2018).

A man-in-the-middle (MITM) attack occurs when an attacker intercepts communication between two parties, potentially altering or eavesdropping on the transmitted data without detection. TLS mitigates such attacks by employing digital certificates, public key cryptography, and authenticated key exchange mechanisms (Stallings, 2017).

TLS Architecture and Security Mechanisms

TLS operates through two primary phases: the *handshake phase* and the *record protocol phase*.

During the handshake phase, the client and server authenticate each other (typically server-side authentication), negotiate cryptographic algorithms, and establish shared session keys. This process relies on asymmetric cryptography and digital certificates issued by trusted Certificate Authorities (CAs). In addition to encryption, TLS ensures data integrity by attaching cryptographic checks to each message, allowing the receiving party to detect any unauthorized modifications.

Once the handshake completes, the record protocol encrypts all application data using symmetric encryption, ensuring both confidentiality and integrity. Message authentication codes (MACs) or authenticated encryption (e.g., AES-GCM) prevent tampering (Rescorla, 2018).

The critical MITM protection mechanism lies in certificate validation. The client verifies that the server's certificate is signed by a trusted CA and matches the intended domain. Without this verification, an attacker could impersonate the server.

Pseudocode for Secure Online Communication with TLS

GitHub repository: https://github.com/Samia1117/mscs-535-residency/tree/main/Project%204

```
class SecureOnlineCommunication {



  void connect(String websiteAddress) {

    // 1. Agree on the communication protocol

    sendToServer("Hello! I'd like to connect securely.");
```

```
// 2. Receive digital certificate from remote server

Certificate cert = receiveFromServer();

// 3. Check certificate is:

// i) issued by a trusted certificate authority

// ii) for the website we are trying to access

// iii) not expired.

if (!isTrustedAuthority(cert.issuer))   abort("Don't trust this site");

if (!cert.name.equals(websiteAddress))  abort("Wrong site — possible impostor");

if (cert.isExpired())                   abort("Certificate expired");



// 4. Key Exchange: create a shared secret (symmetric encryption key)

// Enables fast encryption of data during a session

Number myRandom    = generateRandom();

Number theirRandom = exchangeWith(server, myRandom);

SecretKey sharedKey = combine(myRandom, theirRandom);

// Both sides run combine() and arrive at the same key



// 5. Connection is now considered secure, so use the shared secret to encrypt the bank password

// A man in the middle cannot:
```

```
    // i) decrypt the encrypted message (only the trusted server whose certificate we trusted earlier can)

    // ii) tamper with the message without the trusted server knowing

    // iii) replace the encrypted message with its own malicious message because the MITM does not have

    // the shared secret that only the client and trusted server has

    /// iv) replay the message (in a replay attack)

    sendSecurely(sharedKey, "Hello, bank! My password is 1234");

}

// Send a message securely - encrypted, with tamper stamp and a messageCount to ensure:

// message cannot be decrypted, tampered with, or replayed by an MITM!

void sendSecurely(SecretKey key, String message) {

    String scrambled    = encrypt(message, key);

    String tamperStamp  = sign(scrambled, key);

    int    messageCount = nextMessageNumber();



    send(scrambled, tamperStamp, messageCount);

}

// Receive a message securely

// i) If incoming message number is not the same as the expect message number,

// recognize it as a replay attack and abort the communication
```

```
  // ii) If message was tampered with, abort the communication

  String receiveSecurely(SecretKey key, Packet incoming) {

    if (incoming.number != expectedNumber)  abort("Possible replay attack");

    if (!verifyStamp(incoming, key))       abort("Message was tampered with");



    return decrypt(incoming.scrambled, key);

  }

  // Close the connection and delete any secret generated for the session!

  void disconnect(SecretKey key) {

    send("Goodbye!");

    erase(key);  // Delete the secret — old messages stay safe

  }

}
```

Pseudocode explanation

The `connect()` function is the entry point that initiates a secure connection by first introducing itself to the server, verifying the server's identity certificate (checking it is trusted, matches the intended website, and hasn't expired), and then establishing a shared secret key that both sides independently compute without ever sending it directly across the network. Once connected, `sendSecurely()` handles outgoing messages by scrambling them with the shared

key, attaching a tamper stamp to detect any modifications in transit, and adding a sequential message number to prevent replay attacks. On the receiving end, `receiveSecurely()` does the reverse - it first checks the incoming message number matches what was expected, verifies the tamper stamp hasn't been broken, and only then unscrambles and returns the actual message. Finally, `disconnect()` closes the session by sending a goodbye signal to the server and immediately erasing the shared secret key from memory, ensuring that even if an attacker had recorded all the encrypted traffic, they would have no key left to decrypt it with.

Demonstration of Secure Communication with TLS and Insecure Communication without TLS leading to MITM Attacks

i) Insecure Communication *without* TLS

An example of insecure communication without TLS leading to MITM attack is demonstrated below graphically. When Alice connects to the bank without TLS, her password travels as plain readable text across the network. Anyone sitting between her and the bank, for example someone on the same Wi-Fi, can intercept that traffic and read `password=huunter2` directly, without doing anything sophisticated. There is no verification either, so Samia has no way of knowing she is talking to the real bank and not an impostor. See the illustration below.

1. Alice attempts to communicate with Bank server without TLS (no encryption)

Alice wants to log in to her bank. Her browser sends her username and password over HTTP — plain text, no encryption. Every node between her and the server can read it.

Alice → Bank server `POST /login password=hunter2`

Anyone on Alice's Wi-Fi network can run a packet sniffer and passively read every byte. No active interference needed.

2. Attacker performs ARP spoofing

The attacker is on the same network (a café Wi-Fi). They send forged ARP packets claiming their MAC address belongs to the gateway. Now all of Alice's traffic is silently routed through the attacker's machine.

Attacker → Alice `ARP: gateway = my MAC`

Attacker → Bank server `ARP: Alice = my MAC`

Alice sees no change. Her traffic now flows: Alice → Attacker → Bank. She is completely unaware.

3. Attacker reads credentials in flight

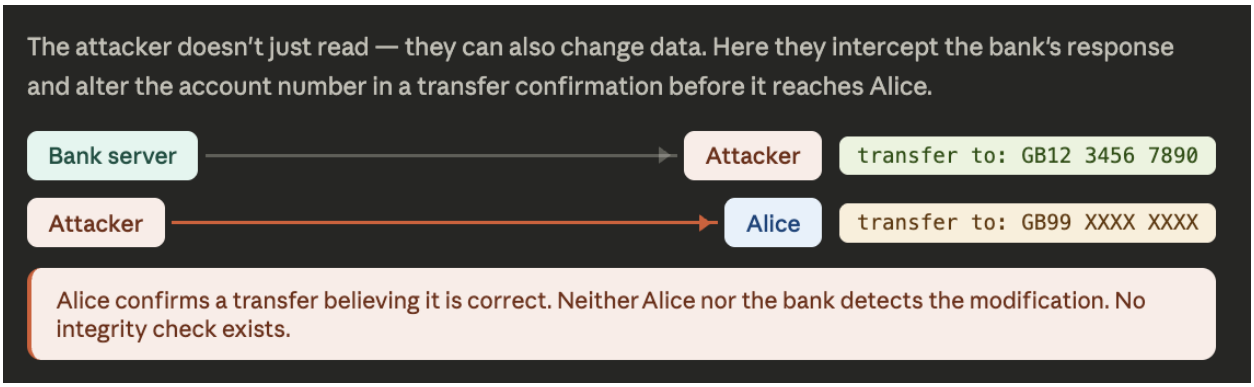
Alice submits her login. The request passes through the attacker's machine. Because there is no encryption, the attacker reads the username and password in plain text before forwarding the request to the real server.

Alice → Attacker `password=hunter2 STOLEN`

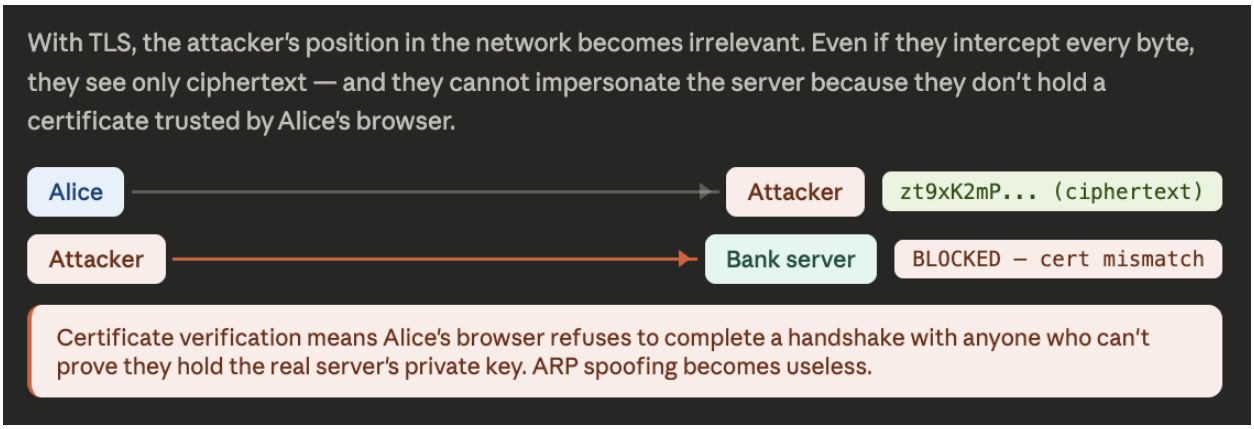
Attacker → Bank server `password=hunter2 (forwarded)`

The bank never knows anything is wrong — Alice successfully logs in. The attacker now has full credentials.

4. Attacker modifies the server's response



5. Why TLS would have prevented the above

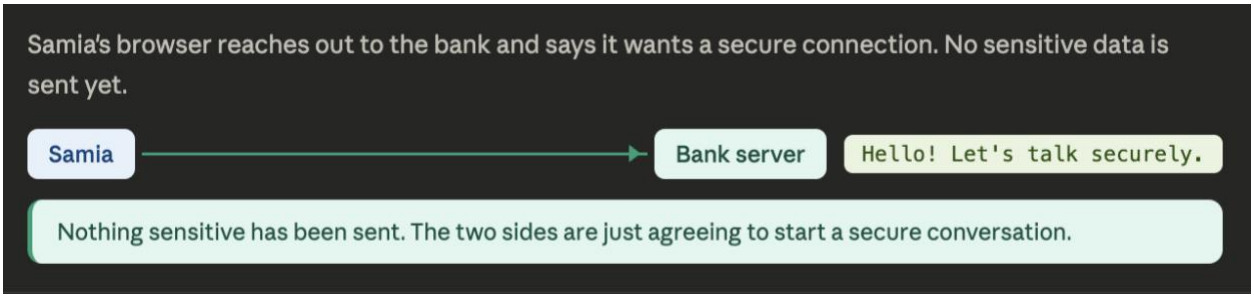


ii) Secure Communication with TLS

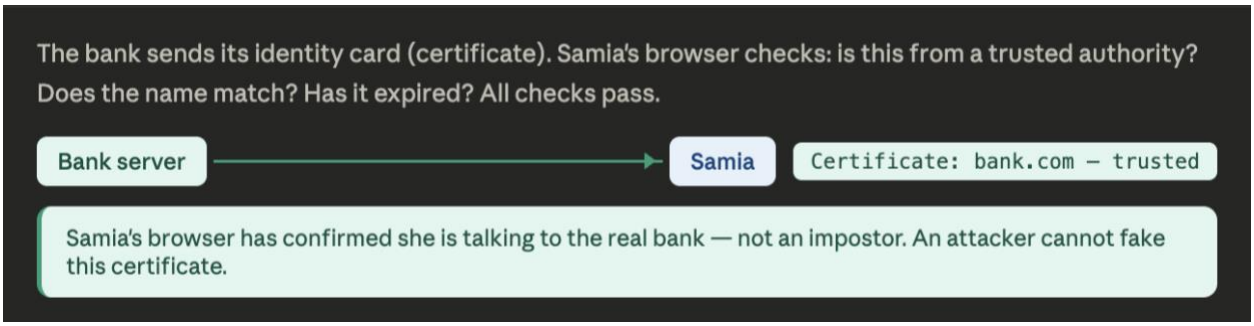
Secure communication using TLS is demonstrated in the graphic below. When TLS is used, three things happen before any sensitive data is sent. First, the bank proves its identity via a certificate, so Samia knows she is talking to the real server. Second, both sides establish a shared secret key without ever sending it across the network. Third, Samia's password is scrambled with that key before transmission - so even if an attacker intercepts the packet, all they see is something like `zt9xK2m...`, which is useless without the key. The connection is also stamped on every message, so any tampering in transit is immediately detected. Below is a high-level

pseudocode representation of a secure client-server communication system using TLS, explicitly designed to prevent MITM attacks.

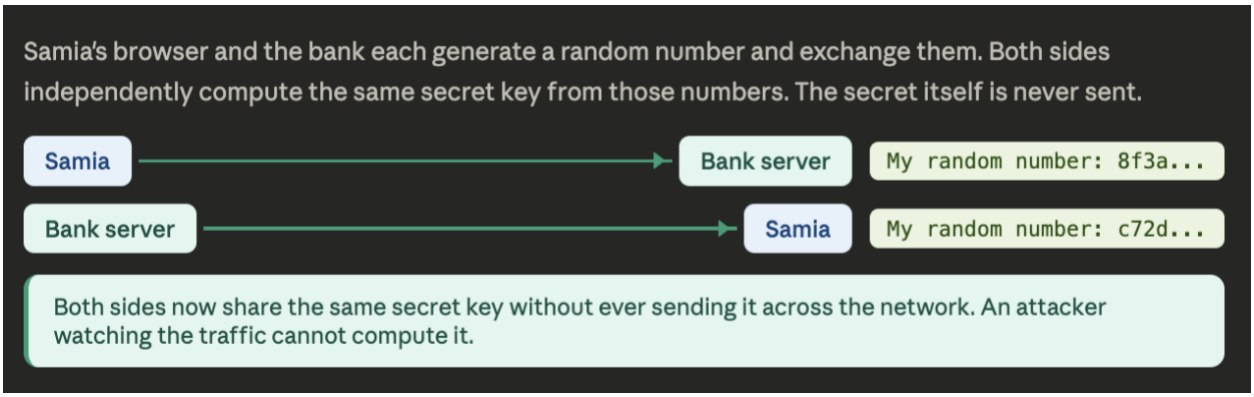
1. Samia communicates with her Bank server *with* TLS (encrypted communication)



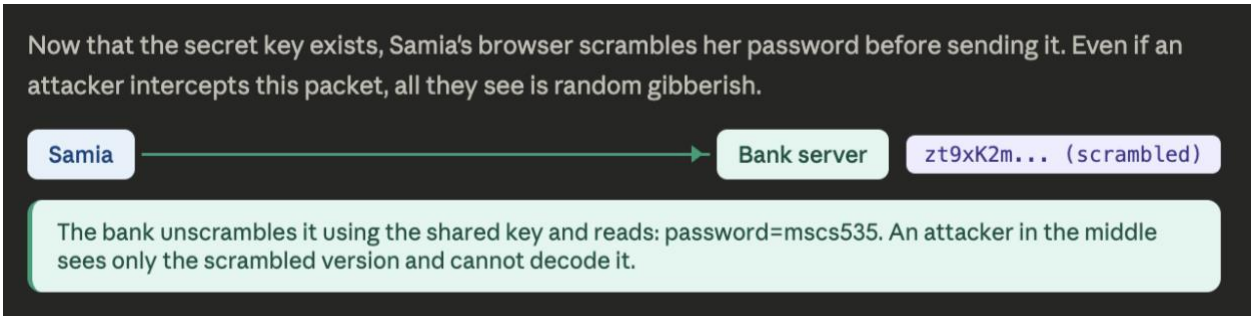
2. Bank proves it is who it says it is



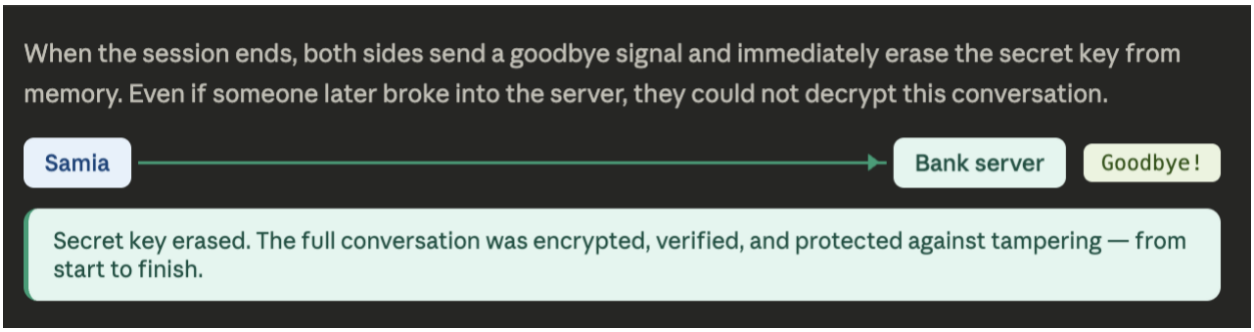
3. Both sides create a shared secret key



4. Samia's password that is sent is scrambled



5. Connection closes safely



Discussion

While TLS provides strong protection against MITM attacks, its effectiveness depends heavily on proper implementation. For example, disabling certificate validation or accepting self-signed certificates without verification introduces vulnerabilities. Similarly, outdated TLS versions or weak cipher suites can undermine security.

In real-world systems, additional safeguards such as certificate pinning, HTTP Strict Transport Security (HSTS), and secure DNS (DNSSEC) further enhance resistance to MITM attacks. Developers must also ensure proper library usage rather than implementing cryptographic primitives manually.

Conclusion

TLS remains the cornerstone of secure online communication, providing robust protection against MITM attacks through authentication, encryption, and integrity guarantees. The pseudocode presented demonstrates how proper certificate validation and secure key exchange mechanisms can be integrated into a client-server model. Ultimately, secure implementation practices are as critical as the protocol itself in ensuring system security.

GitHub repository: https://github.com/Samia1117/mscs-535-residency/tree/main/Project%204

References

Rescorla, E. (2018). *The Transport Layer Security (TLS) Protocol Version 1.3* (RFC 8446). Internet Engineering Task Force. https://doi.org/10.17487/RFC8446

Stallings, W. (2017). *Cryptography and network security: Principles and practice* (7th ed.). Pearson.

Kahn, C., & McGrew, D. (2014). *Authenticated encryption in TLS*. Internet Engineering Task Force.